TruncProof: A Guardrail for LLM-based JSON Generation under Token-Length Constraints

Yoshio Kato* and Shuhei Tarashima
NTT DOCOMO BUSINESS, Inc., Japan

arXiv GitHub

LLM-based JSON generation is widely used for integration with **external** systems.

It is difficult for usual LLM to reliably enforce **grammatically valid** JSON output.

➡ **Grammar-Constrained Generation** restricts the LLM's vocabulary at each generation step based on their syntax parsers.

You are a helpful assistant... Please output only JSON.

max_new_tokens=*N*

```
{
  "userID": "A001",
  "lastname": "Kato",
  "firstname": "Y
```

Many GCG methods support the JSON output, but they can't stop within *max_new_tokens*

*How to avoid truncation under a **strict token limit** without rollback?*

Method

- Rewrite JSON specification in **LL(1) grammar**, which supports enumeration of valid continuations.
- Estimate future token cost of the continuations.

Configuration

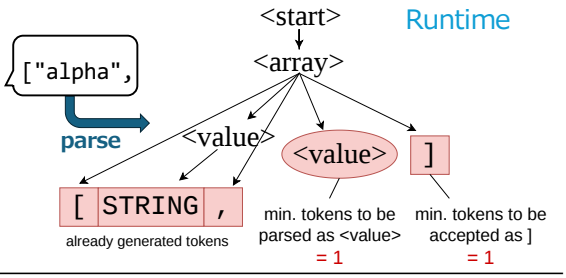
```
?value: object
| array
| STRING
| number
| "true" -> true
| "false" -> false
| "null" -> null
object: "{" [member ("," member)*] "}"
member: STRING ":" value
…
```

Precomputation

Minimum tokens to be accepted as
- STRING /".*"/
- NUMBER /-?[0-9]+/

Minimum tokens to be parsed as
- <array> = []
- <object> = {"a":0}

Runtime



Experiments

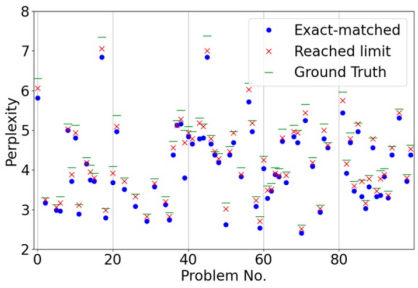
Text-to-JSON task (JSON-Mode-Eval) with **restricting #tokens** to check if methods can reduce whitespace.

- Adhered to JSON grammar **strictly**.
- Ours + Beam Search / Monte-Carlo Tree Search worked well, while others not.

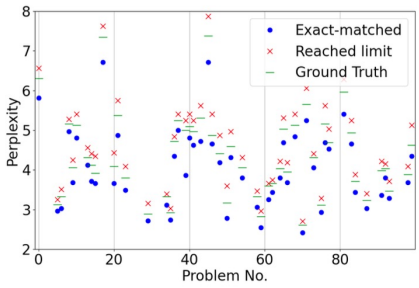
		Gemma2-2B					Llama2-7B-Chat-HF				
		Accuracy (%)					Accuracy (%)				
Method	Decoding	Syntax	Schema	Exact-match	Time (ms)		Syntax	Schema	Exact-match	Time (ms)	
No constraint	Greedy	1	1	0	21.8		2	2	0	17.6	
+*prompt*	Greedy	8	8	4			2	2	0		
Outlines [8]	Greedy	36	33	22	458.7	(+436.9)	18	13	4	72.2	(+54.6)
	BS	4	4	2	4347.8	(+4326.0)	10	8	4	598.8	(+581.2)
Outlines+*prompt*	Greedy	17	17	8			19	17	5		
SynCode [11]	Greedy	4	3	0	23.5	(+1.7)	11	10	4	18.4	(+0.8)
	BS	1	1	0	54.0	(+32.2)	6	6	4	58.7	(+41.1)
	MCTS	4	4	0	438.6	(+416.8)	8	8	4	183.5	(+165.9)
SynCode+*prompt*	Greedy	6	6	1			16	14	5		
XGrammar [12] †	Greedy	5	5	3	**22.1**	**(+0.3)**	11	9	2	**18.3**	**(+0.7)**
	BS	1	1	0	34.3	(+12.5)	5	3	2	32.5	(+14.9)
	MCTS	5	5	2	293.3	(+271.5)	9	8	3	175.1	(+157.5)
XGrammar+*prompt*	Greedy	8	8	4			16	13	3		
Ours	Greedy	**100**	62	21	25.7	(+3.9)	**100**	51	2	19.0	(+1.4)
	BS	**100**	85	37	60.8	(+39.0)	**100**	67	29	37.0	(+19.4)
	MCTS	**100**	**86**	**58**	518.1	(+496.3)	**100**	**70**	**41**	209.2	(+191.6)

Why BS/MCTS worked?

- BS/MCTS selects the candidate with the best perplexity.
- We found the perplexity of truncated outputs were still better than that of the ground truth.
- TruncProof rejects such locally optimal candidates.



(a) SynCode [11]



(b) TruncProof (Ours)

● PPL 6.06
Exact Matched **but long**

```
{
  "ssid": "OfficeNetSecure",
  "securityProtocol": "WPA2-Enterprise",
  "bandwidth": "1300 Mbps"
}
```

✖ PPL 6.08
Reached limit in SynCode

```
{
  "ssid": "OfficeNetSecure",
  "securityProtocol": "WPA2-Enterprise",
  "bandwidth": "1
```

– PPL 6.28 (Ground Truth)

```
{"ssid": "OfficeNetSecure", "securityProtocol":
 "WPA2-Enterprise", "bandwidth": "1300 Mbps"}
```

✖ PPL 6.57
Reached limit in Ours

```
{
  "ssid": "OfficeNetSecure",
  "securityProtocol": "WPA2-Enterprise",
  "bandwidth": "" }
```

Failure cases and ground truth